



INTELIGÊNCIA ARTIFICIAL APLICADA

INSTITUTO NTA

FASE 0 - AGENTE EM CÓDIGO PURO

Aula **0.1**

Setup do projeto e camada agnóstica de provider

Uma função única que fala com Claude e com GPT. Trocar de modelo vira mudar uma variável, não reescrever código.

Curso: Engenharia de IA Aplicada - Agentes e RAG em Produção

Projeto: Núcleo de Agentes Operacionais

Duração alvo: ~1h30 · 25 min teoria + 55 min código + 10 min registro

Pré-requisitos: Python 3.10+, Git, chave de API na Anthropic e na OpenAI

1. Objetivo de aprendizagem

Ao final desta aula você será capaz de fazer uma chamada de LLM crua nos dois principais providers e escrever uma camada fina que abstrai as diferenças de formato, de modo que trocar de modelo seja mudar uma variável, não reescrever código.

2. Por que esta aula importa

Todo agente, por mais sofisticado, no fundo é um loop **while** em volta de uma chamada de LLM. Se você depende de um único provider, três coisas te machucam em produção: o preço muda, o modelo é descontinuado, ou o provider tem um incidente e seu sistema inteiro cai junto. Um engenheiro sênior nunca acopla a lógica de negócio a um SDK específico.

A camada que você vai construir hoje é o que torna o Núcleo vendável. Amanhã um cliente exige que os dados não passem pela OpenAI: você troca uma variável e roda tudo no Claude. Essa portabilidade não é luxo, é requisito. É o primeiro exemplo do princípio que atravessa o curso: **a lógica antes do framework**.

3. Teoria (bloco Ler, ~25 min)

3.1 Uma chamada de LLM é sempre a mesma conversa

Por baixo de qualquer SDK, você manda três coisas para o modelo e recebe texto de volta:

- **Instrução de sistema (system):** quem o modelo é e como deve agir. É a memória procedural do agente.
- **Histórico de mensagens (messages):** a conversa até agora, uma lista de turnos com um papel (role) e um conteúdo (content).
- **Parâmetros (params):** ajustes da geração, como o teto de tokens de saída e, em modelos mais antigos, a temperatura.

Os papéis fundamentais são **user** (o humano ou o sistema falando com o modelo) e **assistant** (o que o modelo já respondeu antes). O system é uma quarta peça, tratada à parte.

Analogia. Pense num ator recebendo uma cena. O system é a direção (você é um detetive cansado, desconfiado de todos). As messages são as falas anteriores. Os params são o quanto ele pode improvisar. Você entrega isso e ele devolve a próxima fala.

3.2 Anthropic e OpenAI dizem o mesmo, com sotaques diferentes

O conceito é idêntico nos dois. O formato, não. Três diferenças importam hoje:

- **Onde vai o system.** Na Anthropic é um parâmetro separado, no topo da chamada. Na OpenAI é mais uma mensagem na lista, com role system, geralmente a primeira.
- **O teto de saída.** Na Anthropic, max_tokens é obrigatório. Na OpenAI é opcional, e nos modelos novos o parâmetro passou a se chamar max_completion_tokens.

- **Onde está a resposta.** Na Anthropic vem em `resposta.content[0].text`. Na OpenAI vem em `resposta.choices[0].message.content`.

3.3 Um cuidado de 2026: sampling nos modelos novos

Antes você mandava `temperature=0.7` para controlar a aleatoriedade. Nos modelos de fronteira de 2026 (Claude Sonnet 5, Fable 5), o raciocínio interno é sempre ligado e vários parâmetros de sampling passaram a ser rejeitados: mandar um valor não padrão de temperatura pode retornar erro. A lição de engenharia: não assuma que todo parâmetro existe em todo modelo. Nossa camada deixa esses ajustes opcionais e, por padrão, nem os envia.

3.4 A decisão de design da nossa camada

Precisamos de um formato interno único, que a gente traduz para cada provider. A escolha natural é espelhar o formato da Anthropic, porque ele separa system de messages, o que é mais limpo:

- Internamente, messages é uma lista de `{"role": "user" ou "assistant", "content": "..."}.`
- system é um argumento separado, uma string.
- A função `chat(provider, messages, system=...)` recebe isso e adapta para o SDK certo.

Para a OpenAI, adaptar significa: se houver system, insira-o como primeira mensagem da lista. Escolhemos o formato mais estrito como padrão e afrouxamos para o mais permissivo, nunca o contrário.

3.5 Sobre nomes de modelo

Nomes de modelo mudam rápido e devem morar no `.env`, nunca chumbados no código. Na Anthropic, os IDs atuais seguem o formato sem sufixo de data, como `claude-sonnet-5`, `claude-opus-4-8` e `claude-sonnet-4-6`, e cada ID mapeia para um snapshot fixo. Na OpenAI, confirme no painel deles qual modelo da geração atual você tem acesso e coloque a string exata no `.env`. Não decore nome de modelo: configure.

4. Mão na massa (bloco Codar, ~55 min)

Vamos criar o repositório do Núcleo e a camada de provider. Todo o resto do curso constrói sobre isto.

Onde cada arquivo mora (a estrutura do projeto)

Desde já o código do produto vive num pacote chamado `nucleo/`. Isso mantém a organização profissional e evita uma migração dolorosa mais tarde. Ao final desta aula o repositório fica assim:

```
nucleo-agentes/
|-- .env                # suas chaves (NUNCA vai pro Git)
|-- .env.example        # modelo do .env, esse vai pro Git
|-- .gitignore
|-- requirements.txt
|-- conftest.py         # prepara os testes (fica na raiz de proposito)
|-- nucleo/            # o pacote do produto
|   |-- __init__.py
|   |-- config.py       # le o .env e os nomes de modelo
|   |-- providers.py    # camada agnostica: a funcao chat()
|-- tests/             # espelha o pacote
|   |-- test_providers.py # os testes com pytest
```

Repare que não há nenhum script solto de demonstração aqui. O código que serve só para ver funcionando é descartável: roda num rascunho e se apaga, para o projeto ficar o mais limpo possível. Só entra no repositório o que é parte do produto ou o que testa o produto.

Os imports usam o pacote: dentro do `nucleo`, o `providers.py` faz `from nucleo import config`; de fora, faz-se `from nucleo.providers import chat`. O `conftest.py` fica na raiz para que o `pytest` enxergue o pacote `nucleo`.

Cada fase seguinte cria uma subpasta dentro de `nucleo/` quando o assunto ganha corpo. Assim, todo código novo já tem endereço previsível. Este é o mapa de para onde o Núcleo cresce:

```
nucleo/
|-- config.py, providers.py  Fase 0   (ja existem)
|-- agent/                  Fase 0   loop e registry (Aula 0.3+)
|-- tools/                  Fase 0   crm, agenda, ticket (Aula 0.5+)
|-- graph/                  Fase 1   grafo LangGraph (state, nodes)
|-- agents/                 Fase 2   supervisor e especialistas
|-- mcp/                    Fase 2   seu servidor MCP
|-- memory/                 Fase 2   memoria do agente
|-- context/                Fase 2   gestao de contexto
|-- eval/                   Fase 3   avaliacao e guardrails
|-- api/                    Fase 4   FastAPI (+ Dockerfile e k8s/ na raiz)
```

A regra. Uma subpasta nasce quando o assunto passa de dois ou três arquivos. O `src/` (o chamado `src-layout`) só entra na Fase 4, ao empacotar para deploy, porque exige instalar o projeto (`pip install -e .`). Por ora, o pacote `nucleo/` na raiz basta e não pede instalação.

Passo 1 - Estrutura do projeto e ambiente isolado

Um venv isola as dependências para não conflitar com outros projetos. Criamos a pasta e o ambiente antes de qualquer código.

```
mkdir nucleo-agentes &&& cd nucleo-agentes

python -m venv .venv
source .venv/bin/activate      # no Windows: .venv\Scripts\activate

git init

# criar o pacote do produto e a pasta de testes
mkdir nucleo tests
touch nucleo/__init__.py      # marca a pasta como pacote Python
```

Resultado esperado: o prompt do terminal mostra (.venv) no início, e você tem a pasta nucleo/ com um __init__.py vazio, que é o que transforma a pasta num pacote importável.

Passo 2 - Segredos fora do código, desde o primeiro commit

Chave de API em código é o erro que vaza credencial no GitHub. Criamos o .env para os segredos e o .gitignore para garantir que ele nunca seja commitado.

Arquivo **.gitignore**:

```
.venv/
.env
__pycache__/
*.pyc
.pytest_cache/
.ipynb_checkpoints/
scratch/
```

Arquivo **.env** (preencha com suas chaves reais):

```
ANTHROPIC_API_KEY=sk-ant-sua-chave-aqui
OPENAI_API_KEY=sk-sua-chave-openai-aqui

ANTHROPIC_MODEL=claude-sonnet-5
OPENAI_MODEL=gpt-5.1
```

Confirme os nomes: os IDs da Anthropic você valida na doc de modelos; o da OpenAI, no painel deles. Se gpt-5.1 não existir na sua conta, troque pela string que aparece lá. Errar o nome do modelo dá erro claro (model not found), fácil de diagnosticar.

Resultado esperado: git status mostra o .gitignore como novo, mas não mostra o .env. Se o .env aparecer, o .gitignore está errado.

Passo 3 - Instalar os SDKs

```
pip install anthropic openai python-dotenv

pip freeze > requirements.txt
```

Passo 4 - Carregar a configuração

Um módulo único que lê o .env e expõe as configurações, evitando os.getenv espalhado. Arquivo **nucleo/config.py**:

```
import os
from dotenv import load_dotenv

# load_dotenv lê o arquivo .env e joga as variaveis pro ambiente.
load_dotenv()

# os.getenv(nome, padrao): pega a variavel; se faltar, usa o padrao.
ANTHROPIC_MODEL = os.getenv("ANTHROPIC_MODEL", "claude-sonnet-5")
OPENAI_MODEL = os.getenv("OPENAI_MODEL", "gpt-5.1")

# Sem padrao aqui de proposito: se faltar a chave, paramos com um
# erro claro, em vez de quebrar la na frente sem explicacao.
if not os.getenv("ANTHROPIC_API_KEY"):
    raise RuntimeError("Falta ANTHROPIC_API_KEY no .env")
if not os.getenv("OPENAI_API_KEY"):
    raise RuntimeError("Falta OPENAI_API_KEY no .env")
```

Passo 5 - A chamada da Anthropic, isolada

Uma função que só sabe falar com a Anthropic. Recebe o formato interno e devolve texto puro. Comece o **nucleo/providers.py**:

```
# SDKs oficiais de cada provider.
from anthropic import Anthropic
from openai import OpenAI

# Nossa config (nomes de modelo), que mora no pacote nucleo.
from nucleo import config

# Um cliente de cada. Eles leem a chave do ambiente sozinhos.
_anthropic = Anthropic()
_openai = OpenAI()

def _chamar_anthropic(messages, system=None, max_tokens=1024):
    # Monta os argumentos da chamada num dicionario.
    kwargs = {
        "model": config.ANTHROPIC_MODEL,      # qual modelo usar
        "max_tokens": max_tokens,             # teto de saida (obrigatorio)
        "messages": messages,                 # o historico da conversa
    }
    # Na Anthropic o system e um parametro a parte; so vai se existir.
    if system:
        kwargs["system"] = system
    # Faz a chamada de verdade e recebe a resposta.
    resposta = _anthropic.messages.create(**kwargs)
    # A resposta vem como lista de blocos; pegamos o texto do 1o.
    return resposta.content[0].text
```

Passo 6 - A chamada da OpenAI, com a tradução do system

A função irmã. O ponto-chave é transformar o system separado numa mensagem role system no início da lista. Adicione ao **nucleo/providers.py**:

```
def _chamar_openai(messages, system=None, max_tokens=1024):
    # Copiamos a lista para não alterar a original de quem chamou.
    mensagens_openai = list(messages)
    # Na OpenAI o system NÃO é separado: vira a 1ª mensagem da lista.
    if system:
        mensagens_openai = [{"role": "system", "content": system}] + mensagens_openai

    # Mesma ideia da Anthropic, com nomes de campo diferentes.
    resposta = _openai.chat.completions.create(
        model=config.OPENAI_MODEL,
        messages=mensagens_openai,
        max_completion_tokens=max_tokens,  # nome novo nos modelos atuais
    )
    # Aqui a resposta mora noutro caminho, não em .content[0].text.
    return resposta.choices[0].message.content
```

Se a sua conta OpenAI reclamar de `max_completion_tokens`, troque por `max_tokens`. Essa é exatamente a diferença de formato que a camada existe para esconder do resto do código.

Passo 7 - A função `chat()` agnóstica

O coração da aula. Uma única porta de entrada. Quem chama não sabe qual provider está por baixo. Adicione ao `nucleo/providers.py`:

```
def chat(provider, messages, system=None, max_tokens=1024):
    # Uma porta só: escolhe a função certa pelo nome do provider.
    if provider == "anthropic":
        return _chamar_anthropic(messages, system, max_tokens)
    if provider == "openai":
        return _chamar_openai(messages, system, max_tokens)
    # Nome desconhecido falha na hora, com mensagem clara.
    raise ValueError(f"Provider desconhecido: {provider}")
```

Passo 8 - Ver funcionando (código descartável)

Para ver a portabilidade acontecendo você não cria um arquivo no projeto. Abra um notebook temporário, ou um rascunho em `scratch/scratch.py` (o `.gitignore` já ignora essa pasta), cole o trecho abaixo, rode, observe e apague. Esse código não entra no repositório: a função dele é te mostrar a coisa funcionando uma vez.

```
# rascunho descartável - rode e apague, não faz parte do projeto
from nucleo.providers import chat

system = "Você é um assistente objetivo. Responda em uma frase."
messages = [
    {"role": "user", "content": "Explique o que é um agente de IA."}
]

for provider in ["anthropic", "openai"]:
    resposta = chat(provider, messages, system=system)
    print(f"\n[{provider}]")
    print(resposta)
```

Rode a célula (ou o rascunho). Resultado esperado: duas respostas, uma sob `[anthropic]` e outra sob `[openai]`, ambas em uma frase. Mesmo código, dois cérebros. Depois apague o rascunho: o que

garante que a portabilidade continua funcionando não é este trecho, é o teste automatizado do próximo passo.

Passo 9 - O teste automatizado com pytest

Aquele trecho descartável chama a API de verdade e gasta tokens; serve para ver, não para garantir. Para o dia a dia, testamos a nossa lógica de tradução com pytest, trocando os clientes por dublês. Não testamos a qualidade da resposta do LLM (isso é avaliação, vem na Fase 3), testamos o código determinístico.

Dois conceitos aparecem aqui. Um **dublê** (fake) é um objeto falso que imita o SDK, grava com o que foi chamado e devolve uma resposta pronta, sem tocar na internet. O **monkeypatch** é uma ferramenta do pytest que, só durante o teste, troca o cliente real pelo dublê.

Arquivo **conftest.py** na raiz (roda antes dos testes e faz o pytest enxergar o pacote nucleo):

```
import os

# setdefault so define se a variavel ainda nao existe: nao
# sobrescreve um valor real que voce ja tenha no ambiente.
os.environ.setdefault("ANTHROPIC_API_KEY", "test-key")
os.environ.setdefault("OPENAI_API_KEY", "test-key")
os.environ.setdefault("ANTHROPIC_MODEL", "claude-sonnet-5")
os.environ.setdefault("OPENAI_MODEL", "gpt-5.1")
```

Arquivo **tests/test_providers.py** (um dos cinco casos, comentado):

```
from types import SimpleNamespace
import pytest
from nucleo import providers

def _fake_openai(texto):
    # Imita a forma da resposta da OpenAI.
    return SimpleNamespace(
        choices=[SimpleNamespace(message=SimpleNamespace(content=texto))])

def test_openai_transforma_system_em_primeira_mensagem(monkeypatch):
    chamadas = {} # guarda com o que o SDK foi chamado

    # Dublê do endpoint: grava os argumentos e devolve algo pronto.
    class FakeCompletions:
        def create(self, **kwargs):
            chamadas.update(kwargs)
            return _fake_openai("ok")

    # Troca, so neste teste, o cliente real pelo dublê.
    monkeypatch.setattr(providers._openai, "chat",
        SimpleNamespace(completions=FakeCompletions()))

    providers.chat("openai", [{"role": "user", "content": "oi"}],
        system="seja breve")

    # O system tem que ter virado a 1a mensagem da lista.
    enviadas = chamadas["messages"]
    assert enviadas[0] == {"role": "system", "content": "seja breve"}
    assert enviadas[1] == {"role": "user", "content": "oi"}
```

Rode com **pip install pytest** e depois **pytest -v**. O arquivo completo cobre cinco casos: provider desconhecido, system separado na Anthropic, system virando mensagem na OpenAI, ausência de system e não mutação da lista original. Resultado esperado: 5 passed.

Passo 10 - Primeiro commit

```
git status          # confirme que .env NAO aparece
git add .
git commit -m "Aula 0.1: setup e camada agnostica de provider"
```

5. Portabilidade

O que você construiu é um padrão universal chamado **adapter** (adaptador): uma interface interna estável, com tradutores por trás para cada serviço externo. Onde ele reaparece:

- **LangChain / LangGraph** (Fase 1) fazem isso por você com `init_chat_model` ou `ChatAnthropic / ChatOpenAI`. Você já vai entender o que eles resolvem.
- **LiteLLM** é uma biblioteca cuja razão de existir é exatamente esta camada, para dezenas de providers.
- **MCP** (Fase 2) leva a mesma ideia para as ferramentas: uma interface padrão para o agente falar com qualquer integração.

A lição portátil: sempre que um serviço externo pode mudar, coloque um adaptador entre ele e a sua lógica.

6. Checkpoint (Terminei quando)

Critério: a mesma função me responde com Claude e com GPT trocando uma variável.

- O trecho descartável (num notebook ou rascunho) retorna duas respostas coerentes, uma por provider.
- Comente "openai" na lista do loop e rode de novo: só o Claude responde, sem tocar em mais nada.
- Troque `ANTHROPIC_MODEL` no `.env` por outro ID válido (ex.: `claude-opus-4-8`) e rode: funciona sem editar código Python.
- `pytest -v` mostra 5 passed.

7. Erros comuns e troubleshooting

- **.env foi para o Git.** Rode `git rm --cached .env`, conserte o `.gitignore` e recommitte. Se a chave subiu para um remoto, revogue e gere outra.
- **authentication_error.** Chave errada ou com espaço. Confira que `load_dotenv()` roda antes de instanciar os clientes.
- **model not found.** O nome no `.env` não existe na sua conta. Valide na doc da Anthropic ou no painel da OpenAI.
- **Erro de sampling.** Modelo novo recusando temperature. Nossa camada não envia isso por padrão; se você adicionou, torne condicional ao modelo.
- **max_tokens vs max_completion_tokens na OpenAI.** Se der erro de parâmetro, troque o nome na `_chamar_openai`.
- **resposta.content[0] estoura na Anthropic.** Resposta vazia ou com outro tipo de bloco. Garanta `max_tokens` suficiente; resposta robusta vem na Aula 0.6.

8. Ponte para a próxima aula

Você tem uma boca (falar com o LLM). Falta dar mãos ao modelo. Na Aula 0.2, Tool use, o mecanismo cru, vamos ver por baixo como o modelo pede para executar uma ferramenta, reusando a `chat()` que acabamos de construir.